МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Методы проектирования баз данных

Отчет о выполнении работы на тему

«Программирование и настройка СУБД»

Студент,
группы 5130201/40102 Адиатуллин Т. Р. _____

Руководитель,
доцент к.т.н., доцент Попов С. Г. _____

«____»_____ 2026 г.

Санкт-Петербург, 2026

Реферат

Содержание

Введение **4**

Постановка задачи **5**

1 Представления **6**
1.1 Запрос для создание представления 6
1.2 Выполнение запросов к представлению 7

2 Триггеры **11**
2.1 Создание таблицы статистики 11
2.2 Триггеры для таблицы автомобилей 11
2.3 Триггеры для тревожных событий 13
2.4 Триггер для парковочных сессий 14
2.5 Графическое описание схемы базы данных 14
2.6 Проверка работы триггеров 16
2.7 Перечень тестовых примеров 22

3 Права пользователей **24**
3.1 Постановка задачи 24
3.2 Назначение прав доступа 24
3.3 Проверка прав пользователей 25

4 Функции и процедуры **32**

5 Транзакции **33**
5.1 Постановка задачи 33
5.2 Выполнение задания 33
5.3 Результаты выполнения 34
5.4 Вывод 34

Заключение **37**

Список использованных источников **38**

Приложение А **39**

Приложение Б **40**

Введение

Постановка задачи

1. Представления

В данном разделе создаётся представление по ранее созданной базе данных предметной области «Геомониторинг автопарка каршеринга» и выполняются запросы к нему.

Постановка задачи

- Создать представление на основе таблиц базы данных по предметной области Геомониторинг автопарка каршеринга.
- Написать и выполнить простой запрос к созданному представлению.
- Написать и выполнить сложный запрос к созданному представлению.

1.1. Запрос для создание представления

Для каждого автомобиля подсчитать количество тревожных событий и число парковочных зон, в которых этот автомобиль парковался.

```sql
CREATE VIEW car_stats AS
SELECT
  c.car_id,
  c.reg_number,
  COUNT(DISTINCT a.alert_event_id) AS alert_count,
  COUNT(DISTINCT pz.parking_zone_id) AS parking_count
FROM car c
LEFT JOIN alert_event a ON c.car_id = a.car_id
LEFT JOIN parking_session ps ON c.car_id = ps.car_id
LEFT JOIN parking_zone pz ON ps.parking_zone_id =
   pz.parking_zone_id
GROUP BY c.car_id, c.reg_number;
```

Listing 1: Создание представления `car_stats`

	car_id	reg_number	alert_count	parking_cou…
1	1	A001AA77	3	15
2	2	A002AA78	3	19
3	3	A003AA79	2	12
4	4	A004AA80	3	19
5	5	A005AA81	1	10
6	6	A006AA77	1	19
7	7	A007AA78	1	10
8	8	A008AA79	3	10
9	9	A009AA80	2	15
10	10	A010AA81	3	11
11	11	A011AA77	2	19
12	12	A012AA78	3	13
13	13	A013AA79	1	13
14	14	A014AA80	1	10
15	15	A015AA81	2	14

Рис. 1: Содержимое представления `car_stats`

В качестве основной таблицы выбрана таблица `car`. `LEFT JOIN` позволяют включить в представление каждый автомобиль, в том числе автомобиль без тревожных событий и парковочных сессий. Использование `COUNT(DISTINCT ...)` необходимо, поскольку соединение событий и парковочных сессий может образовать повторяющиеся строки.
Реляционная формула представления:

$$\gamma_{\substack{\text{car_id, reg_number;}\\ \text{COUNT(alert_event_id)}\to\text{alert_count;}\\ \text{COUNT(parking_zone_id)}\to\text{parking_count}}} \Big(\text{car} \bowtie (left)_{\text{car_id}} \text{alert_event} \bowtie (left)_{\text{car_id}} \text{parking_session} \bowtie (left)_{\text{parking_zone_id}} \text{parking_zone} \Big)$$

1.2. Выполнение запросов к представлению

Простой запрос

Вывести автомобили, которые парковались более чем в N разных зонах и имеют хотя бы два тревожных события.

```sql
SELECT
  c.reg_number,
  c.alert_count,
  c.parking_count
FROM
  car_stats c
WHERE
  c.parking_count > 17
  AND
  c.alert_count > 1;
```

Listing 2: Простой запрос

Результат выполнения простого запроса представлен на рисунке 2.

	reg_number	alert_count	parking_count
1	A002AA78	3	19
2	A004AA80	3	19
3	A011AA77	2	19
4	A016AA77	3	19
5	A024AA80	3	19
6	A027AA78	2	20
7	A041AA77	2	18
8	A053AA79	3	19
9	A067AA78	3	18
10	A072AA78	3	19
11	A073AA79	3	20
12	A088AA79	3	19
13	A089AA80	3	19
14	A090AA81	3	20
15	A091AA77	2	20
16	A093AA79	2	18
17	A097AA78	3	18
18	A099AA80	2	18
19	A110AA81	3	20
20	A121AA77	2	18

Рис. 2: Результат выполнения простого запроса к представлению

Время выполнения запроса:
Execution time: 0,207ms
Planning time: 0,291ms

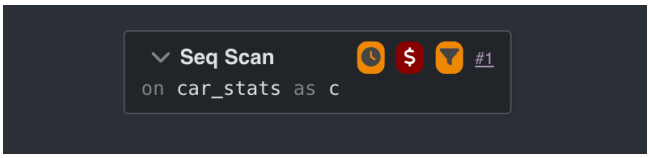


Рис. 3: Последовательность выполнения простого запроса

Таблица **car_stats** читается последовательно. Во время чтения проверяются условия на количество парковочных зон и тревожных событий, после чего в результат попадают только подходящие автомобили.
Реляционная формула запроса:

$$\pi_{\text{reg_number, alert_count, parking_count}}\Big(\sigma_{\text{parking_count}>17 \,\wedge\, \text{alert_count}>1}\big(\text{car_stats}\big)\Big)$$

Сложный запрос

Для выполнения сложного запроса к созданному представлению был сформулирован запрос: вывести автомобили, которые парковались менее чем в N разных зонах и имеют необработанные тревожные события заданного типа.

```sql
SELECT
  DISTINCT
  c.reg_number,
  c.alert_count
FROM
  car_stats c
JOIN
  alert_event a ON c.car_id = a.car_id
JOIN
  alert_event_process_status s ON a.status_id = s.status_id
JOIN
  alert_event_type t ON a.alert_event_type_id =
   t.alert_event_type_id
WHERE
  c.parking_count < 18
  AND
  s.name = 'Новое'
  AND
  t.name = 'Потеря связи';
```

Listing 3: Сложный запрос

Результат выполнения сложного запроса представлен на рисунке 4. В результате выполнения было выведено 19 автомобилей.

	reg_number	alert_count
1	A010AA81	3
2	A021AA77	1
3	A034AA80	2
4	A040AA81	2
5	A044AA80	1
6	A045AA81	2
7	A058AA79	3
8	A063AA79	3
9	A132AA78	2
10	A138AA79	2
11	A141AA77	3
12	A157AA78	3
13	A173AA79	3
14	A189AA80	3
15	A197AA78	3

Рис. 4: Результат выполнения сложного запроса к представлению

Время выполнения запроса:
Execution time: 0,564ms
Planning time: 1,5ms

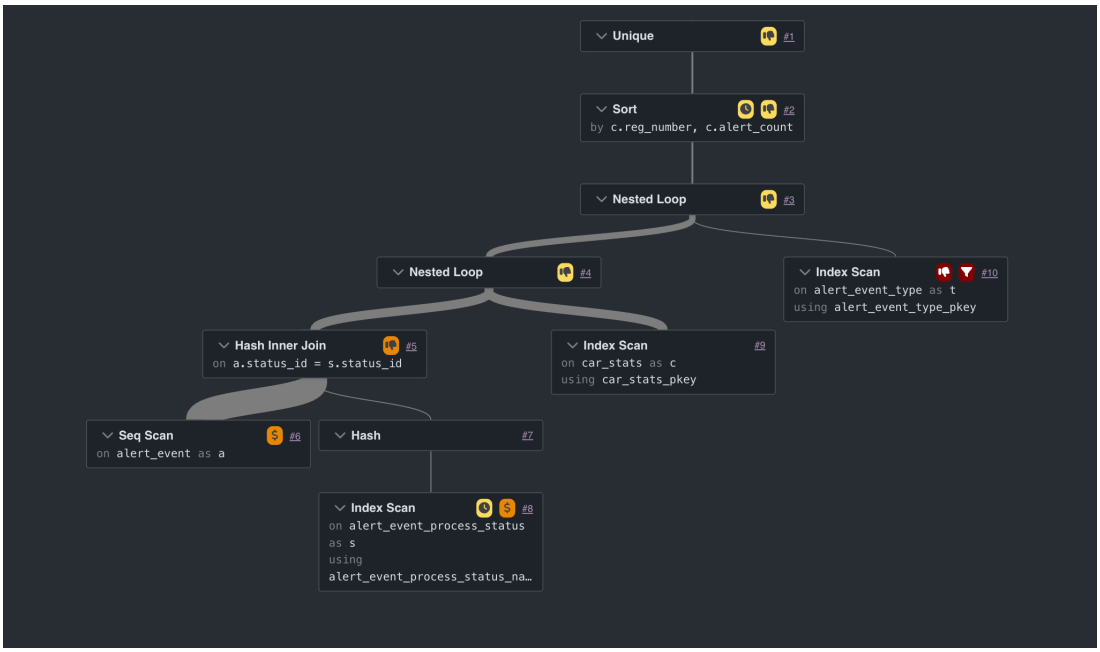


Рис. 5: Последовательность выполнения сложного запроса

Сначала последовательно читается таблица **alert_event**, а по индексу находятся статусы обработки с именем «Новое». Эти таблицы соединяются через Hash Join. Затем для каждого подходящего события по первичному ключу находятся строка в **car_stats** и его тип события. В конце результат сортируется, а оператор **Unique** удаляет повторяющиеся автомобили.
Реляционная формула запроса:

$$\pi_{\text{reg_number, alert_count}}\Big(\sigma_{\text{parking_count}<18 \,\wedge\, \text{s.name}=\text{«Новое»} \,\wedge;\text{t.name}=\text{«Потеря связи»}}\Big(\big(\text{car_stats} \bowtie_{\text{car_id}} \text{alert_event} \bowtie_{\text{status_id}} \text{alert_event_process_status} \bowtie_{\text{alert_event_type_id}} \text{alert_event_type}\big)\Big)\Big)$$

2. Триггеры

Постановка задачи

На основе представления **car_stats** создаётся таблица с теми же показателями. Триггеры должны поддерживать согласованность её данных при изменении таблиц **car**, **alert_event** и **parking_session**.

2.1. Создание таблицы статистики

Таблица с описание атрибутов новой таблицы

Таблица 1: Статистика автомобилей – car_stats

Статистика автомобилей – car_stats							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	авто_id	car_id	SERIAL	-	PK	-	NOT NULL
2	гос_номер	reg_number	VARCHAR(30)	-	-	-	NOT NULL
3	кол-во событий	alert_count	BIGINT	0	-	-	NOT NULL
4	кол-во парковок	parking_count	BIGINT	0	-	-	NOT NULL

Код создания таблицы

Сначала представление переименовывается в **car_stats_view**, после чего создаётся таблица **car_stats** и заполняется данными из представления. Порядок операций важен: представление и таблица не могут иметь одинаковое имя.

```
ALTER VIEW car_stats RENAME TO car_stats_view;

CREATE TABLE car_stats (
    car_id        INT PRIMARY KEY REFERENCES car(car_id) ON DELETE
   CASCADE,
    reg_number    VARCHAR(9) NOT NULL,
    alert_count   BIGINT NOT NULL DEFAULT 0,
    parking_count BIGINT NOT NULL DEFAULT 0
);

INSERT INTO car_stats (car_id, reg_number, alert_count,
   parking_count)
SELECT car_id, reg_number, alert_count, parking_count
FROM car_stats_view;
```

Listing 4: Создание и начальное заполнение таблицы статистики

2.2. Триггеры для таблицы автомобилей

При добавлении автомобиля в таблице статистики создаётся строка с нулевыми счётчиками. При изменении регистрационного номера обновляется его

копия в **car_stats**. Перед удалением автомобиля удаляется соответствующая строка статистики.

```
CREATE OR REPLACE FUNCTION car_stats_change()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO car_stats (car_id, reg_number, alert_count,
   parking_count)
        VALUES (NEW.car_id, NEW.reg_number, 0, 0);
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE car_stats
        SET reg_number = NEW.reg_number
        WHERE car_id = NEW.car_id;
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        DELETE FROM car_stats
        WHERE car_id = OLD.car_id;
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER car_stats_after_insert
AFTER INSERT ON car
FOR EACH ROW EXECUTE FUNCTION car_stats_change();

CREATE TRIGGER car_stats_after_reg_number_update
AFTER UPDATE OF reg_number ON car
FOR EACH ROW
WHEN (OLD.reg_number IS DISTINCT FROM NEW.reg_number)
EXECUTE FUNCTION car_stats_change();

CREATE TRIGGER car_stats_before_delete
BEFORE DELETE ON car
FOR EACH ROW EXECUTE FUNCTION car_stats_change();
```

Listing 5: Функция и триггеры для таблицы `car`

2.3. Триггеры для тревожных событий

При добавлении тревожного события счётчик **alert_count** автомобиля увеличивается на единицу, а при удалении — уменьшается.

```
CREATE OR REPLACE FUNCTION alert_event_stats_change()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE car_stats
        SET alert_count = alert_count + 1
        WHERE car_id = NEW.car_id;
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE car_stats
        SET alert_count = GREATEST(alert_count - 1, 0)
        WHERE car_id = OLD.car_id;
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER alert_event_after_insert
AFTER INSERT ON alert_event
FOR EACH ROW EXECUTE FUNCTION alert_event_stats_change();

CREATE TRIGGER alert_event_after_delete
AFTER DELETE ON alert_event
FOR EACH ROW EXECUTE FUNCTION alert_event_stats_change();

-- update
CREATE TRIGGER alert_event_after_update
AFTER UPDATE ON alert_event
FOR EACH ROW
EXECUTE FUNCTION alert_event_stats_change();
```

Listing 6: Функция и триггеры для таблицы `alert_event`

2.4. Триггер для парковочных сессий

Счётчик **parking_count** хранит число различных зон, поэтому при удалении одной сессии уменьшать его можно только тогда, когда у автомобиля больше не осталось сессий в этой же зоне. Функция на листинге 7 проверяет это условие до удаления строки.

```
CREATE OR REPLACE FUNCTION parking_session_stats_delete()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM parking_session
        WHERE car_id = OLD.car_id
          AND parking_zone_id = OLD.parking_zone_id
          AND parking_session_id <> OLD.parking_session_id
    ) THEN
        UPDATE car_stats
        SET parking_count = GREATEST(parking_count - 1, 0)
        WHERE car_id = OLD.car_id;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER parking_session_before_delete
BEFORE DELETE ON parking_session
FOR EACH ROW
EXECUTE FUNCTION parking_session_stats_delete();
```

Listing 7: Триггер удаления парковочной сессии

Отдельный триггер на удаление **parking_zone** не требуется. Внешний ключ **parking_session.parking_zone_id** настроен с правилом **ON DELETE CASCADE**; следовательно, удаление зоны удаляет связанные парковочные сессии, а триггер **parking_session_before_delete** корректирует статистику. Это предотвращает двойное уменьшение счётчика для одной зоны.

2.5. Графическое описание схемы базы данных

Графическое описание схемы базы данных с новой таблицей и триггерами приведено на рисунке 6.

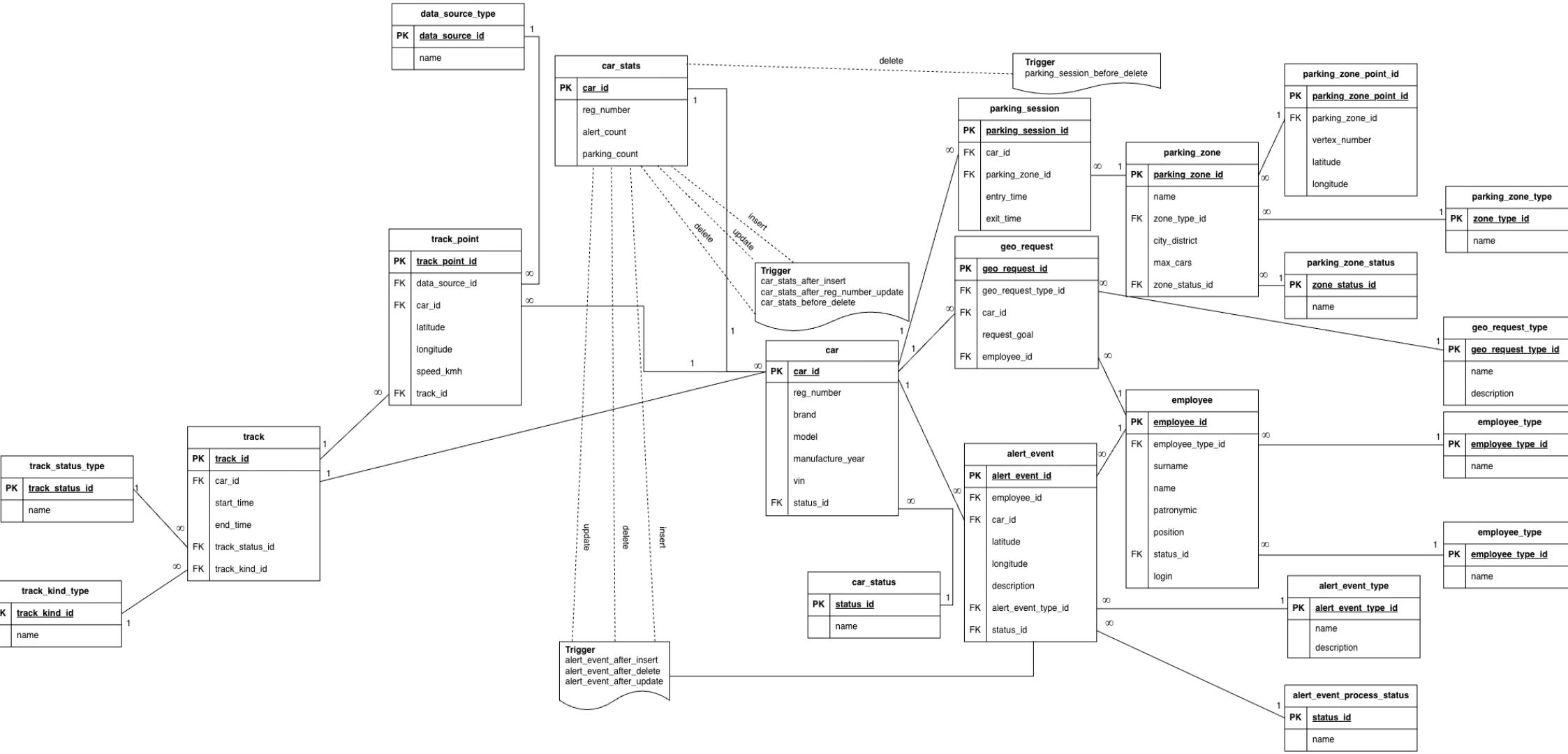


Рис. 6: Схема базы данных на английском языке

2.6. Проверка работы триггеров

Перед проверкой триггеров выполняем команду `BEGIN;` для начала транзакции. После проверки всех триггеров выполняется команда `ROLLBACK;` для отмены изменений. Чтобы не засорять базу данных тестовыми данными.

Проверка триггеров таблицы автомобилей

Сначала добавляется тестовый автомобиль.

```
INSERT INTO car (
    reg_number, brand, model, manufacture_year, vin, car_status_id
)
VALUES (
    'TTT77778', 'Test', 'Trigger', 2026, 'VINTRIGGER0000000',
    (SELECT car_status_id FROM car_status LIMIT 1)
);
```

Listing 8: Добавление тестового автомобиля

После добавления в таблице статистики должна появиться строка с нулевыми счётчиками.

```
SELECT cs.*
FROM car_stats cs
JOIN car c ON c.car_id = cs.car_id
WHERE c.vin = 'VINTRIGGER0000000';
```

Listing 9: Проверка статистики после добавления автомобиля

	car_id	reg_number	alert_cou…	parking_co…
1	202	T777TT78	0	0

Рис. 7: Таблица `car_stats` после добавления тестового автомобиля

Далее изменяется регистрационный номер автомобиля.

```
UPDATE car
SET reg_number = 'TTT88878'
WHERE vin = 'VINTRIGGER0000000';
```

Listing 10: Изменение регистрационного номера

```
SELECT cs.*
FROM car_stats cs
JOIN car c ON c.car_id = cs.car_id
WHERE c.vin = 'VINTRIGGER0000000';
```

Listing 11: Проверка статистики после изменения номера

	car_id	reg_number	alert_cou…	parking_co…
1	202	T888TT78	0	0

Рис. 8: Таблица `car_stats` после изменения регистрационного номера автомобиля

В конце тестовый автомобиль удаляется.

```
DELETE FROM car
WHERE vin = 'VINTRIGGER0000000';
```

Listing 12: Удаление тестового автомобиля

```
SELECT *
FROM car_stats
WHERE reg_number = 'TTT88878';
```

Listing 13: Проверка удаления строки статистики



Рис. 9: Таблица `car_stats` после удаления тестового автомобиля

Проверка триггеров тревожных событий

Выбирем существующий идентификатор автомобиля и сначала зафиксируем его исходное значение **alert_count**.

```
SELECT car_id, alert_count
FROM car_stats
WHERE car_id = 1;
```

Listing 14: Исходная статистика тревожных событий

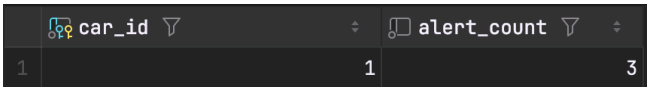


Рис. 10: Таблица `car_stats` до добавления тревожного события

Для выбранного автомобиля добавим тестовое тревожное событие. Полученный идентификатор используется в запросе удаления.

```
INSERT INTO alert_event (
    car_id, alert_event_type_id, status_id, description
)
VALUES (
    1,
    1,
    1,
    'Тестовое тревожное событие'
)
RETURNING alert_event_id;
```

Listing 15: Добавление тревожного события

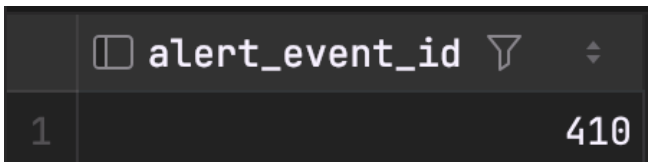


Рис. 11: Идентификатор добавленного тревожного события

```sql
SELECT car_id, alert_count
FROM car_stats
WHERE car_id = 1;
```

Listing 16: Проверка статистики после добавления события

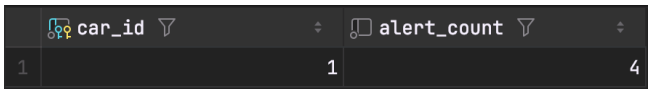


Рис. 12: Таблица `car_stats` после добавления тревожного события

```sql
DELETE FROM alert_event
WHERE alert_event_id = 410;
```

Listing 17: Удаление тестового тревожного события

```sql
SELECT car_id, alert_count
FROM car_stats
WHERE car_id = 1;
```

Listing 18: Проверка статистики после удаления события

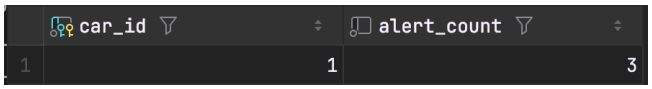


Рис. 13: Таблица `car_stats` после удаления тестового тревожного события

Также если сделать Update и перекинуть событие на другой автомобиль, то счётчик уменьшится у старого автомобиля и увеличится у нового.

```sql
SELECT * FROM car_stats
WHERE car_id = 200;
```

Listing 19: Статистика автомобиля до переноса тревожного события

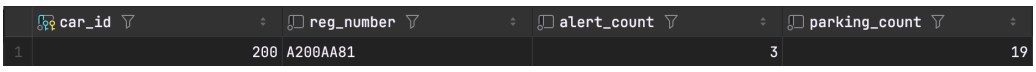


Рис. 14: Таблица `car_stats` до переноса тревожного события на другой автомобиль

```sql
UPDATE alert_event
SET car_id = 203
WHERE alert_event_id = 408;
```

Listing 20: Перенос тревожного события на другой автомобиль

```sql
SELECT * FROM car_stats
WHERE car_id = 203;
```

Listing 21: Проверка статистики нового автомобиля после переноса события

	car_id	reg_number	alert_count	parking_count
1	203	T777TT78	1	0

Рис. 15: Таблица `car_stats` после переноса тревожного события на другой автомобиль

```sql
SELECT * FROM car_stats
WHERE car_id = 200;
```

Listing 22: Проверка статистики прежнего автомобиля после переноса события

	car_id	reg_number	alert_count	parking_count
1	200	A200AA81	2	19

Рис. 16: Таблица `car_stats` после переноса тревожного события на другой автомобиль

Проверка триггера парковочных сессий

Найдем автомобиль с двумя или более сессиями в одной зоне. Из результата запомним `car_id` и `parking_zone_id`.

```sql
SELECT
    ps.car_id,
    ps.parking_zone_id,
    COUNT(*) AS session_count
FROM parking_session ps
GROUP BY ps.car_id, ps.parking_zone_id
HAVING COUNT(*) >= 2
LIMIT 1;
```

Listing 23: Поиск данных для проверки парковочных сессий

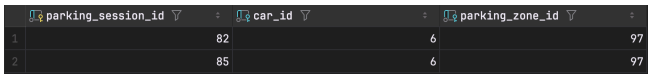
	car_id	parking_zone_id	session_count
1	6	97	2

Рис. 17: Результат поиска автомобиля с двумя и более сессиями в одной зоне

После подстановки найденных значений отдельным запросом выведем идентификаторы сессий. Один из них необходимо оставить для последнего удаления.

```sql
SELECT
    parking_session_id,
    car_id,
    parking_zone_id
FROM parking_session
WHERE car_id = 6
  AND parking_zone_id = 97;
```

Listing 24: Выбор сессий найденного автомобиля в зоне



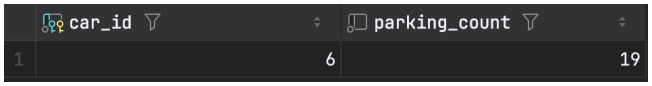
	parking_session_id	car_id	parking_zone_id
1	82	6	97
2	85	6	97

Рис. 18: Результат запроса: идентификаторы парковочных сессий выбранного автомобиля в выбранной зоне

Сначала удаляются все сессии выбранной зоны, кроме одной: счётчик не должен измениться. После удаления последней сессии значение **parking_count** должно уменьшиться на единицу.

```sql
SELECT car_id, parking_count
FROM car_stats
WHERE car_id = 6;
```

Listing 25: Исходная статистика парковок



	car_id	parking_count
1	6	19

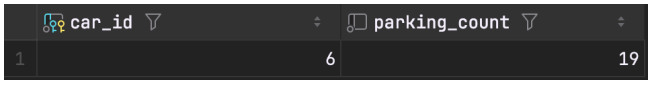
Рис. 19: Исходная статистика парковок

```sql
DELETE FROM parking_session
WHERE car_id = 6
  AND parking_zone_id = 97
  AND parking_session_id <> 82;
```

Listing 26: Удаление всех сессий зоны, кроме одной

```sql
SELECT car_id, parking_count
FROM car_stats
WHERE car_id = 6;
```

Listing 27: Проверка после удаления не последней сессии



	car_id	parking_count
1	6	19

Рис. 20: Результат запроса: `parking_count` не изменился после удаления всех сессий зоны, кроме одной

```sql
DELETE FROM parking_session
WHERE parking_session_id = 82;
```

Listing 28: Удаление последней сессии в зоне

```sql
SELECT car_id, parking_count
FROM car_stats
WHERE car_id = 6;
```

Listing 29: Проверка после удаления последней сессии

	car_id	parking_count
1	6	18

Рис. 21: Результат запроса: `parking_count` уменьшился на 1 после удаления последней сессии

Проверка каскадного удаления парковочной зоны

Отдельный триггер на таблицу **parking_zone** не создаётся. Внешний ключ в таблице **parking_session** имеет правило **ON DELETE CASCADE**, поэтому при удалении зоны PostgreSQL удаляет её парковочные сессии. Для каждой удаляемой сессии запускается триггер **parking_session_before_delete**, который обновляет **parking_count**.

Для проверки выбирается зона, в которой имеется хотя бы одна парковочная сессия. Необходимо сохранить значения `car_id` и `parking_zone_id`.

```sql
SELECT
    car_id,
    parking_zone_id
FROM parking_session
GROUP BY car_id, parking_zone_id
LIMIT 1;
```

Listing 30: Поиск данных для проверки каскадного удаления

	car_id	parking_zone_id
1	1	3

Рис. 22: Результат запроса: идентификаторы автомобиля и парковочной зоны для проверки каскадного удаления

Перед удалением зоны фиксируется исходная статистика выбранного автомобиля.

```sql
SELECT car_id, parking_count
FROM car_stats
WHERE car_id = 1;
```

Listing 31: Исходная статистика автомобиля

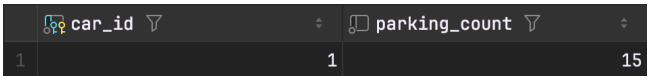


Рис. 23: Таблица `car_stats` до удаления парковочной зоны

```sql
DELETE FROM parking_zone
WHERE parking_zone_id = 3;
```

Listing 32: Удаление парковочной зоны

После удаления зоны связанных с ней сессий в таблице **parking_session** быть не должно.

```sql
SELECT *
FROM parking_session
WHERE parking_zone_id = 3;
```

Listing 33: Проверка каскадного удаления сессий



Рис. 24: Парковочные сессии удалённой зоны отсутствуют в таблице `parking_session`

В результате значение **parking_count** выбранного автомобиля должно уменьшиться на единицу.

```sql
SELECT car_id, parking_count
FROM car_stats
WHERE car_id = 1;
```

Listing 34: Проверка статистики после удаления зоны

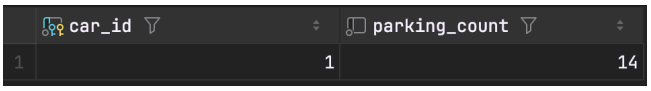


Рис. 25: Таблица `car_stats` после удаления парковочной зоны: `parking_count` уменьшился на 1

2.7. Перечень тестовых примеров

В таблице приведены все тестовые примеры из подраздела проверки работы триггеров. Описание каждого примера включает исходное состояние, выполняемое изменение и проверяемый результат. Проверки выполняются внутри транзакции, которая завершается командой `ROLLBACK;`.

№	Описание проверяемого состояния
1	После добавления автомобиля в таблице `car_stats` появляется связанная строка с нулевыми счётчиками.
2	После изменения регистрационного номера автомобиля проверяется, что связанная строка статистики сохраняется и остаётся связанной с автомобилем.
3	После удаления автомобиля проверяется, что соответствующая строка статистики также удалена.
4	После добавления тревожного события проверяется что произошло увеличение счётчика значение `alert_count` на 1.
5	После удаления добавленного тревожного события проверяется уменьшение `alert_count` автомобиля на 1 относительно прошлого значения.
6	При переносе тревожного события с автомобиля с идентификатором A на автомобиль B проверяется, что у первого счётчик `alert_count` уменьшился на 1, а у второго увеличился на 1.
7	Для автомобиля, имеющего не менее двух сессий в одной парковочной зоне, удаляются все сессии, кроме одной. Проверяется, что `parking_count` не изменился.
8	После удаления оставшейся последней сессии автомобиля в этой зоне проверяется уменьшение `parking_count` на 1.
9	Перед удалением парковочной зоны фиксируется статистика автомобиля. После удаления зоны проверяется каскадное удаление связанных сессий и уменьшение `parking_count` на число удалённых сессий (в приведённом примере — на 1).

3. Права пользователей

3.1. Постановка задачи

В работе настроено разграничение доступа для двух пользователей базы данных. Пользователи **А** и **Б** могут читать таблицу `car_stats`. Пользователь А имеет только это право. Пользователь Б, помимо чтения статистики, может читать и изменять исходные таблицы `car`, `alert_event`, `parking_session` и `parking_zone`. При изменении этих таблиц связанные счётчики в `car_stats` обновляются созданными ранее триггерами.

3.2. Назначение прав доступа

Пользователям выданы права на подключение к базе данных и использование схемы `public`. Оба пользователя могут читать таблицу `car_stats`. Пользователю Б также разрешено читать и изменять исходные таблицы и пользоваться последовательностями, формирующими идентификаторы. Функции триггеров настроены с режимом `SECURITY DEFINER` и фиксированной схемой поиска, чтобы они могли обновлять счётчики в `car_stats` при изменении исходных данных. SQL-команды настройки приведены в листинге 35.

```
CREATE USER user_a WITH PASSWORD '123';
CREATE USER user_b WITH PASSWORD '123';

GRANT CONNECT ON DATABASE db TO user_a, user_b;
GRANT USAGE ON SCHEMA public TO user_a, user_b;

-- Пользователи А и Б могут читать таблицу статистики
GRANT SELECT ON car_stats TO user_a, user_b;

-- Пользователь Б читает и изменяет исходные таблицы
GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLE car, alert_event, parking_session, parking_zone
TO user_b;

-- для пользователя Б разрешаем автоинкрементные id
GRANT USAGE, SELECT ON SEQUENCE
  car_car_id_seq,
  alert_event_alert_event_id_seq,
  parking_session_parking_session_id_seq,
  parking_zone_parking_zone_id_seq
TO user_b;

ALTER FUNCTION car_stats_change() SECURITY DEFINER  SET
   search_path = public;

ALTER FUNCTION alert_event_stats_change() SECURITY DEFINER SET
   search_path = public;

```

```
27 ALTER FUNCTION parking_session_stats_delete() SECURITY DEFINER SET
     search_path = public;
```

Listing 35: Создание пользователей и назначение прав

3.3. Проверка прав пользователей

Проверка выполнена в двух консольных сеансах, подключённых к одной базе данных: в первом использовалась учётная запись пользователя А, во втором — пользователя Б. Это позволило проверить, какие операции доступны каждой учётной записи, и убедиться, что триггеры обновляют статистику при изменении исходных данных.

Чтение таблиц. Пользователи А и Б выполнили запрос к `car_stats` и получили первые пять строк со статистикой автомобилей. Пользователь Б также смог прочитать таблицу `car`. Результаты проверки чтения статистики приведены в таблице 3, чтения таблицы автомобилей — на рисунке 26.

```
db=> SELECT car_id, reg_number
FROM car
LIMIT 5;
 car_id | reg_number
--------+------------
      1 | A001AA77
      2 | A002AA78
      3 | A003AA79
      4 | A004AA80
      5 | A005AA81
(5 rows)
```

Рис. 26: Пользователь Б читает таблицу car

Добавление тревожного события. Попытка пользователя А добавить строку в `alert_event` была отклонена из-за отсутствия права `INSERT`. Пользователь Б выполнил такую вставку успешно. Этот результат приведён на рисунке 27.



Рис. 27: Вставка в alert_event запрещена пользователю А и разрешена пользователю Б

Для проверки триггера была выполнена ещё одна вставка пользователем Б. В этом сеансе команда вернула ID `414`. До вставки значение `alert_count` для автомобиля с ID 1 составляло 5, после вставки оно стало равно 6. Пользователь Б при этом не смог прочитать `car_stats`, а пользователь А увидел обновлённое значение счётчика. Результат показан на рисунке 28.

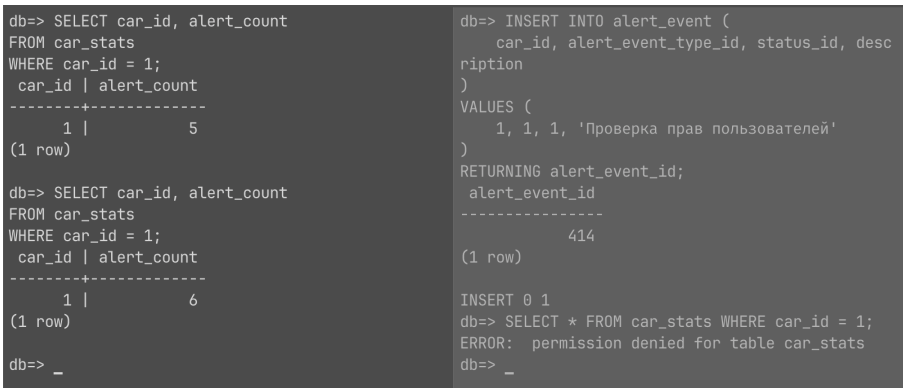


Рис. 28: После вставки тревожного события счётчик изменился с 5 до 6; прямое чтение car_stats пользователю Б недоступно

Удаление тревожного события. Пользователь А попытался удалить событие с ID `414`, но СУБД отказала из-за отсутствия права `DELETE`. Пользователь Б удалил эту запись успешно. После удаления значение `alert_count`, отображаемое пользователю А, изменилось с 6 на 5. Результат приведён на рисунке 29.

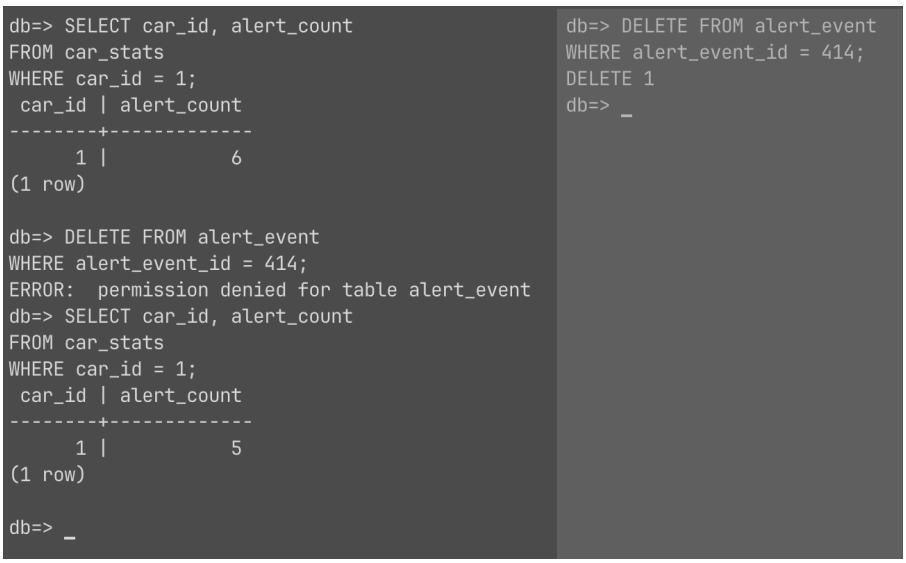


Рис. 29: Пользователю А запрещено удалять событие; пользователь Б удалил запись `alert_event`

Обращение к таблице без выданных прав. Пользователи А и Б выполнили запрос к `alert_event_type`. Поскольку право чтения этой таблицы не выдавалось ни одной учётной записи, обе попытки завершились сообщением `permission denied`. Результат показан на рисунке 30.



Рис. 30: Обеим учётным записям отказано в чтении alert_event_type

Таблица 3: Результаты выполнения тестовых запросов для пользователей `user_a` и `user_b`

№	Текст SQL-запроса	Реакция пользователя **`user_a`**	Реакция пользователя **`user_b`**

1	`SELECT * FROM car_stats LIMIT 5;`	`db=> SELECT * FROM car_stats LIMIT 5;`
		
		`car_id | reg_number | alert_count | parking_count`
		`---------+------------+-------------+---------------`
		`4 | A004AA80 | 3 | 19`
		`5 | A005AA81 | 1 | 10`
		`6 | A006AA77 | 1 | 19`
		`7 | A007AA78 | 1 | 10`
		`8 | A008AA79 | 3 | 10`
		`(5 rows)`
			`db=> SELECT * FROM car_stats LIMIT 5;`
			
			`car_id | reg_number | alert_count | parking_count`
			`---------+------------+-------------+---------------`
			`4 | A004AA80 | 3 | 19`
			`5 | A005AA81 | 1 | 10`
			`6 | A006AA77 | 1 | 19`
			`7 | A007AA78 | 1 | 10`
			`8 | A008AA79 | 3 | 10`
			`(5 rows)`

2	`INSERT INTO car_stats (car_id, reg_number, alert_count, parking_count) VALUES (999999, 'T999TT78', 0, 0);`	`db=> INSERT INTO car_stats (car_id, reg_number, alert_count, parking_count) VALUES (999999, 'T999TT78', 0, 0);`
		
		`ERROR: permission denied for table car_stats`
		
		Проверка результата:
		Запись с car_id = 999999 отсутствует
		`db=> SELECT car_id, reg_number FROM car_stats WHERE car_id = 999999;`
		
		`car_id | reg_number`
		`---------+------------`
		`(0 rows)`
			`db=> INSERT INTO car_stats (car_id, reg_number, alert_count, parking_count) VALUES (999999, 'T999TT78', 0, 0);`
			
			`ERROR: permission denied for table car_stats`
			
			Проверка результата:
			Запись с car_id = 999999 отсутствует
			`db=> SELECT car_id, reg_number FROM car_stats WHERE car_id = 999999;`
			
			`car_id | reg_number`
			`---------+------------`
			`(0 rows)`

3	`UPDATE car_stats SET alert_count = 100 WHERE car_id = 1;`	`db=> UPDATE car_stats SET alert_count = 100 WHERE car_id = 1;`
		
		`ERROR: permission denied for table car_stats`
		
		Проверка результата:
		alert_count остался прежним
		`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
		
		`car_id | alert_count`
		`---------+------------`
		`1 | 5`
		`(1 row)`
			`db=> UPDATE car_stats SET alert_count = 100 WHERE car_id = 1;`
			
			`ERROR: permission denied for table car_stats`
			
			Проверка результата:
			alert_count остался прежним
			`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
			
			`car_id | alert_count`
			`---------+------------`
			`1 | 5`
			`(1 row)`

№	Текст SQL-запроса	Реакция пользователя `user_a`	Реакция пользователя `user_b`

4	`DELETE FROM car_stats WHERE car_id = 1;`	`db=> DELETE FROM car_stats WHERE car_id = 1;`
		`ERROR:  permission denied for table car_stats`
		Проверка результата:
		Строка с car_id = 1 сохранилась
		`db=> SELECT car_id FROM car_stats WHERE car_id = 1;`
		`car_id`
		`-------`
		`1`
		`(1 row)`
			`db=> DELETE FROM car_stats WHERE car_id = 1;`
			`ERROR:  permission denied for table car_stats`
			Проверка результата:
			Строка с car_id = 1 сохранилась
			`db=> SELECT car_id FROM car_stats WHERE car_id = 1;`
			`car_id`
			`-------`
			`1`
			`(1 row)`

5	`INSERT INTO alert_event (car_id, alert_event_type_id, status_id, description) VALUES (1, 1, 1, 'Проверка прав') RETURNING alert_event_id;`	`db=> INSERT INTO alert_event (car_id, alert_event_type_id, status_id, description) VALUES (1, 1, 1, 'Проверка прав') RETURNING alert_event_id;`
		`ERROR:  permission denied for table alert_event`
			`db=> INSERT INTO alert_event (car_id, alert_event_type_id, status_id, description) VALUES (1, 1, 1, 'Проверка прав') RETURNING alert_event_id;`
			`alert_event_id`
			`----------------`
			`416`
			`(1 row)`
			`INSERT 0 1`

6	`SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`	`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
		`car_id | alert_count`
		`--------+-------------`
		`1 | 6`
		`(1 row)`
			`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
			`car_id | alert_count`
			`--------+-------------`
			`1 | 6`
			`(1 row)`

№	Текст SQL-запроса	Реакция пользователя `user_a`	Реакция пользователя `user_b`

7	`UPDATE alert_event SET description = 'Проверка UPDATE' WHERE alert_event_id = 416;`	`db=> UPDATE alert_event SET description = 'Проверка UPDATE прав пользователя Б' WHERE alert_event_id = 416;`
		
		`ERROR:  permission denied for table alert_event`
		
		Проверка результата:
		Счётчик не изменился
		`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
		
		`car_id | alert_count`
		`--------+-------------`
		`1 | 6`
		`(1 row)`
			`db=> UPDATE alert_event SET description = 'Проверка UPDATE прав пользователя Б' WHERE alert_event_id = 416;`
			
			`UPDATE 1`
			
			Проверка результата:
			Описание события изменено; счётчик не изменился
			`db=> SELECT description FROM alert_event WHERE alert_event_id = 416;`
			`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
			
			`description`
			`-----------------------------------------`
			`Проверка UPDATE прав пользователя Б`
			`(1 row)`
			
			`car_id | alert_count`
			`--------+-------------`
			`1 | 6`
			`(1 row)`

8	`DELETE FROM alert_event WHERE alert_event_id = 416;`	`db=> DELETE FROM alert_event WHERE alert_event_id = 416;`
		
		`ERROR:  permission denied for table alert_event`
		
		Проверка результата:
		Событие и счётчик не изменились
		`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
		
		`car_id | alert_count`
		`--------+-------------`
		`1 | 6`
		`(1 row)`
			`db=> DELETE FROM alert_event WHERE alert_event_id = 416;`
			
			`DELETE 1`
			
			Проверка результата:
			Событие удалено; счётчик уменьшился на 1
			`db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;`
			
			`car_id | alert_count`
			`--------+-------------`
			`1 | 5`
			`(1 row)`
			
			`db=> SELECT alert_event_id FROM alert_event WHERE alert_event_id = 416;`
			
			`alert_event_id`
			`----------------`
			`(0 rows)`

9	`SELECT * FROM car LIMIT 3;`	`db=> SELECT * FROM car LIMIT 3;`
		
		`ERROR:  permission denied for table car`
			`db=> SELECT * FROM car LIMIT 3;`
			
			`car_id | reg_number | brand | model | manufacture_year | vin | car_status_id`
			`--------+------------+-------+---------+-----------------+-------------------+--------------`
			`1 | A001AA77 | Lada | Largus | 2014 | X0000000000000001 | 2`
			`2 | A002AA78 | Kia | Sorento | 2026 | X0000000000000002 | 2`
			`3 | A003AA79 | Lada | Vesta | 2016 | X0000000000000003 | 4`
			`(3 rows)`

№	Текст SQL-запроса	Реакция пользователя `user_a`	Реакция пользователя `user_b`
10	`SELECT * FROM alert_event_type LIMIT 3;`		

4. Функции и процедуры

5. Транзакции

5.1. Постановка задачи

Реализовать сценарий взаимодействия двух пользователей с базой данных с помощью транзакций и проверить наличие аномалии при выбранном уровне изоляции. В качестве уровня изоляции выбран `READ COMMITTED`, а в качестве проверяемой аномалии — неповторяемое чтение.

5.2. Выполнение задания

Для проверки используются созданные ранее учётные записи А и Б с правами, описанными в предыдущем разделе. Пользователь А может читать статистику автомобилей, а пользователь Б — добавлять тревожные события. Созданный ранее триггер увеличивает счётчик тревог после вставки события. Эксперимент проводится на существующих данных, новая таблица не создаётся.

Сначала в отдельном сеансе пользователя А запускается транзакция и считывается текущее значение счётчика автомобиля с идентификатором 1. Транзакцию нужно оставить открытой, не выполняя `COMMIT`.

```
BEGIN ISOLATION LEVEL READ COMMITTED;

SELECT car_id, alert_count
FROM car_stats
WHERE car_id = 1;
-- Результат проверки: alert_count = 5
```

Listing 36: Пользователь А: начало транзакции и первое чтение

Пока транзакция А остаётся открытой, в отдельном сеансе пользователя Б добавляется тревожное событие. Команда возвращает идентификатор созданной записи; его нужно сохранить, чтобы удалить именно тестовое событие после проверки. После `COMMIT` изменение счётчика становится доступно другим транзакциям.

```
BEGIN ISOLATION LEVEL READ COMMITTED;

INSERT INTO alert_event (
    car_id, alert_event_type_id, status_id, description
)
VALUES (
    1, 1, 1, 'ЛР: проверка READ COMMITTED'
)
RETURNING alert_event_id;

COMMIT;
```

Listing 37: Пользователь Б: добавление тревожного события

После завершения транзакции Б пользователь А повторяет тот же запрос в своей всё ещё открытой транзакции. Затем транзакция А завершается.

```
SELECT car_id, alert_count
FROM car_stats
WHERE car_id = 1;
-- Результат проверки: alert_count = 6

COMMIT;
```

Listing 38: Пользователь А: повторное чтение и завершение

5.3. Результаты выполнения

Последовательность действий пользователей и полученные результаты приведены в таблице 4.

№	Пользователь user_a	Пользователь user_b
1	Начинает транзакцию READ COMMITTED и читает alert_count = 5. Транзакцию оставляет открытой.	—
2	Ожидает, не завершая транзакцию.	Добавляет строку в alert_event. Триггер увеличивает alert_count; команда возвращает идентификатор события, затем выполняется COMMIT.
3	Повторно читает ту же строку и получает alert_count = 6; завершает транзакцию командой COMMIT.	—
4	—	Удаляет тестовое событие по полученному alert_event_id. Триггер уменьшает счётчик до исходного значения.

Таблица 4: Результаты проверки неповторяемого чтения

Для наглядного подтверждения результаты сеансов приведены отдельно на рисунках 31 и 32.

Для удаления тестовой записи пользователь Б подставляет идентификатор, полученный в листинге 37:

```
DELETE FROM alert_event
WHERE alert_event_id = 418;
-- При повторном запуске подставить ID из результата INSERT
```

Listing 39: Удаление тестового тревожного события

5.4. Вывод

Сценарий подтвердил наличие неповторяемого чтения. Уровень изоляции — READ COMMITTED. Пользователь А в одной транзакции сначала получил значение счётчика 5, а после фиксации изменения пользователем Б — 6. Это

```
db=> BEGIN ISOLATION LEVEL READ COMMITTED;

db=*> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;

 car_id | alert_count
--------+------------
      1 |           5
(1 row)

db=*> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;

 car_id | alert_count
--------+------------
      1 |           6
(1 row)

db=*> COMMIT;
```

Рис. 31: Сеанс пользователя `user_a`: неповторяемое чтение счётчика

```
db=> BEGIN ISOLATION LEVEL READ COMMITTED;

db=*> INSERT INTO alert_event (car_id, alert_event_type_id, status_id, description) VALUES
    (1, 1, 1, 'ЛР: проверка READ COMMITTED') RETURNING alert_event_id;

 alert_event_id
----------------
            418
(1 row)

db=*> COMMIT;

db=> DELETE FROM alert_event WHERE alert_event_id = 418;
DELETE 1

db=> SELECT car_id, alert_count FROM car_stats WHERE car_id = 1;

 car_id | alert_count
--------+------------
      1 |           5
(1 row)
```

Рис. 32: Сеанс пользователя `user_b`: изменение счётчика добавлением тревожного события

возможно, потому что на уровне READ COMMITTED каждый оператор получает новый снимок данных. Поэтому два запроса в одной транзакции могут видеть разные зафиксированные значения.

Заключение

Список литературы

Приложение А «Исходный текст функции»

Приложение Б «Исходный текст процедуры»